

Integration Manual

for S32G CAN_LLCE Driver

Document Number: IM11CAN_LLCEASR4.4 Rev0000R1.0.7 Rev. 1.0

1 Revision History	2
2 Introduction	3
2.1 Supported Derivatives	3
2.2 Overview	4
2.3 About This Manual	4
2.4 Acronyms and Definitions	5
2.5 Reference List	5
3 Building the driver	6
3.1 Build Options	6
3.1.1 GCC Compiler/Assembler/Linker Options	6
3.1.2 GHS Compiler/Assembler/Linker Options	9
3.2 Files required for compilation	11
3.3 Setting up the plugins	14
4 Function calls to module	16
4.1 Function Calls during Start-up	16
4.2 Function Calls during Shutdown	16
4.3 Function Calls during Wake-up	16
5 Module requirements	17
5.1 Exclusive areas to be defined in BSW scheduler	17
5.2 Exclusive areas not available on this platform	19
5.3 Peripheral Hardware Requirements	19
5.4 ISR to Configure within OS - Dependencies	19
5.5 ISR Macro	19
5.5.1 Without an Operating System	19
5.5.2 With an Operating System	20
5.6 Other AUTOSAR modules - dependencies	20
5.7 Data Cache Restriction	20
5.8 User Mode support	20
5.8.1 User Mode configuration in the module	21
5.8.2 User Mode configuration in AutosarOS	21
5.9 Multicore support	22
6 Main API Requirements	24
6.1 Main function calls within BSW scheduler	24
6.2 API Requirements	25
6.3 Calls to Notification Functions, Callbacks, Callouts	25
7 Memory allocation	26



7.1 Sections to be defined in MemMap.h	26
7.2 Linker command file	27
8 Integration Steps	28
9 External assumptions for driver	29



Chapter 1

Revision History

Revision	Date	Author	Description
1.0	29.09.2023	NXP LLCE Team	Prepared for release S32G LLCE AUTOSAR 4.4 Version 1.0.7

Chapter 2

Introduction

- [Supported Derivatives](#)
- [Overview](#)
- [About This Manual](#)
- [Acronyms and Definitions](#)
- [Reference List](#)

This integration manual describes the integration requirements for CAN_43_LLCE Driver for S32G microcontrollers.

2.1 Supported Derivatives

The software described in this document is intended to be used with the following microcontroller devices of NXP Semiconductors:

- s32g274a_bga525
- s32g254a_bga525
- s32g233a_bga525
- s32g234m_bga525
- s32g378a_bga525
- s32g379a_bga525
- s32g398a_bga525
- s32g399a_bga525
- s32g338m_bga525
- s32g339m_bga525
- s32g358a_bga525
- s32g359a_bga525

All of the above microcontroller devices are collectively named as S32.

2.2 Overview

AUTOSAR (AUTomotive Open System ARchitecture) is an industry partnership working to establish standards for software interfaces and software modules for automobile electronic control systems.

AUTOSAR:

- paves the way for innovative electronic systems that further improve performance, safety and environmental friendliness.
- is a strong global partnership that creates one common standard: "Cooperate on standards, compete on implementation".
- is a key enabling technology to manage the growing electrics/electronics complexity. It aims to be prepared for the upcoming technologies and to improve cost-efficiency without making any compromise with respect to quality.
- facilitates the exchange and update of software and hardware over the service life of the vehicle.

2.3 About This Manual

This Technical Reference employs the following typographical conventions:

- **Boldface** style: Used for important terms, notes and warnings.
- *Italic* style: Used for code snippets in the text. Note that C language modifiers such "const" or "volatile" are sometimes omitted to improve readability of the presented code.

Notes and warnings are shown as below:

Note

This is a note.

Warning

This is a warning

2.4 Acronyms and Definitions

Term	Definition
API	Application Programming Interface
AUTOSAR	Automotive Open System Architecture
ASM	Assembler
BSMI	Basic Software Make file Interface
CAN	Controller Area Network
DEM	Diagnostic Event Manager
DET	Development Error Tracer
C/CPP	C and C++ Source Code
VLE	Variable Length Encoding
N/A	Not Applicable
MCU	Micro Controller Unit

2.5 Reference List

#	Title	Version
1	Specification of CAN_43_LLCE Driver	AUTOSAR Release 4.4.0
2	S32G2 Reference Manual	Rev. 5, 05/2022
3	S32G2 Data Sheet	Rev. 5, 05/2022
4	S32G2 Errata Document	S32G2 Mask Set Errata for Mask 0P77B, Rev. 3.1
5	S32G2 Errata Document	S32G2 Mask Set Errata for Mask 1P77B, Rev. 1.1
6	S32G3 Reference Manual	Rev. 2, 09/2022
7	S32G3 Data Sheet	Rev. 2, Draft C, 09/2021
8	S32G3 Errata Document	S32G3 Mask Set Errata for Mask 1P72B, Rev. 2.2

Chapter 3

Building the driver

- [Build Options](#)
- [Files required for compilation](#)
- [Setting up the plugins](#)

This section describes the source files and various compilers, linker options used for building the driver. It also explains the EB Tresos Studio plugin setup procedure.

3.1 Build Options

- [GCC Compiler/Assembler/Linker Options](#)
- [GHS Compiler/Assembler/Linker Options](#)

The RTD driver files are compiled using:

- NXP GCC 9.2.0 20190812 (Build 1649 Revision gaf57174) (for both S32G2 and S32G3)
- Green Hills Multi 7.1.6d / Compiler 2020.1.4 (for both S32G2 and S32G3)

The compiler, assembler, and linker flags used for building the driver are explained below.

The TS_T40D11M10I7R0 part of the plugin name is composed as follows:

- T = Target_Id (e.g. T40 identifies Cortex-M architecture)
- D = Derivative_Id (e.g. D11 identifies S32 platform)
- M = SW_Version_Major and SW_Version_Minor
- I = SW_Version_Patch
- R = Reserved

3.1.1 GCC Compiler/Assembler/Linker Options

3.1.1.1 GCC Compiler Options

Compiler Option	Description
-mcpu=cortex-m7	Targeted ARM processor for which GCC should tune the performance of the code
-mthumb	Generates code that executes in Thumb state
-mlittle-endian	Generate code for a processor running in little-endian mode
-mfpv=fpv5-sp-d16	Specifies the floating-point hardware available on the target
-mfloat-abi=hard	Specifies the floating-point ABI to use. "hard" allows generation of floating-point instructions and uses FPU-specific calling conventions
-std=c99	Specifies the ISO C99 base standard
-Os	Optimize for size. Enables all -O2 optimizations except those that often increase code size
-ggdb3	Produce debugging information for use by GDB using the most expressive format available, including GDB extensions if at all possible. Level 3 includes extra information, such as all the macro definitions present in the program
-Wall	Enables all the warnings about constructions that some users consider questionable, and that are easy to avoid (or modify to prevent the warning), even in conjunction with macros
-Wextra	This enables some extra warning flags that are not enabled by -Wall
-pedantic	Issue all the warnings demanded by strict ISO C. Reject all programs that use forbidden extensions. Follows the version of the ISO C standard specified by the aforementioned -std option
-Wstrict-prototypes	Warn if a function is declared or defined without specifying the argument types
-Wundef	Warn if an undefined identifier is evaluated in an #if directive. Such identifiers are replaced with zero
-Wunused	Warn whenever a function, variable, label, value, macro is unused
-Werror=implicit-function-declaration	Make the specified warning into an error. This option throws an error when a function is used before being declared
-Wsign-compare	Warn when a comparison between signed and unsigned values could produce an incorrect result when the signed value is converted to unsigned.
-Wdouble-promotion	Give a warning when a value of type float is implicitly promoted to double
-fno-short-enums	Specifies that the size of an enumeration type is at least 32 bits regardless of the size of the enumerator values.
-funsigned-char	Let the type char be unsigned by default, when the declaration does not use either signed or unsigned
-funsigned-bitfields	Let a bit-field be unsigned by default, when the declaration does not use either signed or unsigned
-fomit-frame-pointer	Omit the frame pointer in functions that don't need one. This avoids the instructions to save, set up and restore the frame pointer; on many targets it also makes an extra register available.

Compiler Option	Description
-fno-common	Makes the compiler place uninitialized global variables in the BSS section of the object file. This inhibits the merging of tentative definitions by the linker so you get a multiple-definition error if the same variable is accidentally defined in more than one compilation unit
-fstack-usage	This option is only used to build test for generation Ram/↔ Stack size report. Makes the compiler output stack usage information for the program, on a per-function basis
-fdump-ipa-all	This option is only used to build test for generation Ram/↔ Stack size report. Enables all inter-procedural analysis dumps
-c	Stop after assembly and produce an object file for each source file
-DS32XX	Predefine S32XX as a macro, with definition 1
-DS32G2XX	Predefine S32G2XX as a macro, with definition 1
-DS32G3XX	Predefine S32G3XX as a macro, with definition 1
-DS32R45	Predefine S32R45 as a macro, with definition 1
-DGCC	Predefine GCC as a macro, with definition 1
-DUSE_SW_VECTOR_MODE	Predefine USE_SW_VECTOR_MODE as a macro, with definition 1. By default, the drivers are compiled to handle interrupts in Software Vector Mode
-DD_CACHE_ENABLE	Predefine D_CACHE_ENABLE as a macro, with definition 1. Enables data cache initialization in source file system.↔ c under the Platform driver
-DI_CACHE_ENABLE	Predefine I_CACHE_ENABLE as a macro, with definition 1. Enables instruction cache initialization in source file system.c under the Platform driver
-DENABLE_FPU	Predefine ENABLE_FPU as a macro, with definition 1. Enables FPU initialization in source file system.c under the Platform driver
-DMCAL_ENABLE_USER_MODE_SUPPORT	Predefine MCAL_ENABLE_USER_MODE_SUPPORT↔ RT as a macro, with definition 1. Allows drivers to be configured in user mode.

3.1.1.2 GCC Assembler Options

Assembler Option	Description
-X assembler-with-cpp	Specifies the language for the following input files (rather than letting the compiler choose a default based on the file name suffix)
-mcpu=cortexm7	Targeted ARM processor for which GCC should tune the performance of the code
-mfpu=fpv5-sp-d16	Specifies the floating-point hardware available on the target
-mfloat-abi=hard	Specifies the floating-point ABI to use. "hard" allows generation of floating-point instructions and uses FPU-specific calling conventions
-mthumb	Generates code that executes in Thumb state
-c	Stop after assembly and produce an object file for each source file

3.1.1.3 GCC Linker Options

Linker Option	Description
-Wl,-Map,filename	Produces a map file
-T linkerfile	Use linkerfile as the linker script. This script replaces the default linker script (rather than adding to it)
-entry=Reset_Handler	Specifies that the program entry point is Reset_Handler
-nostartfiles	Do not use the standard system startup files when linking
-mcpu=cortexm7	Targeted ARM processor for which GCC should tune the performance of the code
-mthumb	Generates code that executes in Thumb state
-mfpu=fpv5-sp-d16	Specifies the floating-point hardware available on the target
-mfloat-abi=hard	Specifies the floating-point ABI to use. "hard" allows generation of floating-point instructions and uses FPU-specific calling conventions
-mlittle-endian	Generate code for a processor running in little-endian mode
-ggdb3	Produce debugging information for use by GDB using the most expressive format available, including GDB extensions if at all possible. Level 3 includes extra information, such as all the macro definitions present in the program
-lc	Link with the C library
-lm	Link with the Math library
-lgcc	Link with the GCC library

3.1.2 GHS Compiler/Assembler/Linker Options

3.1.2.1 GHS Compiler Options

Compiler Option	Description
-cpu=cortexm7	Selects target processor: Arm Cortex M7
-thumb	Selects generating code that executes in Thumb state
-fpu=vfpv5_d16	Specifies hardware floating-point using the v5 version of the VFP instruction set, with 16 double-precision floating-point registers
-fsingle	Use hardware single-precision, software double-precision FP instructions
-C99	Use (strict ISO) C99 standard (without extensions)
-ghstd=last	Use the most recent version of Green Hills Standard mode (which enables warnings and errors that enforce a stricter coding standard than regular C and C++)
-Osize	Optimize for size
-gnu_asm	Enables GNU extended asm syntax support
-dual_debug	Generate DWARF 2.0 debug information
-G	Generate debug information
-keeptempfiles	Prevents the deletion of temporary files after they are used. If an assembly language file is created by the compiler, this option will place it in the current directory instead of the temporary directory

Compiler Option	Description
-Wimplicit-int	Produce warnings if functions are assumed to return int
-Wshadow	Produce warnings if variables are shadowed
-Wtrigraphs	Produce warnings if trigraphs are detected
-Wundef	Produce a warning if undefined identifiers are used in #if preprocessor statements
-unsigned_chars	Let the type char be unsigned, like unsigned char
-unsigned_fields	Bitfields declared with an integer type are unsigned
-no_commons	Allocates uninitialized global variables to a section and initializes them to zero at program startup
-no_exceptions	Disables C++ support for exception handling
-no_slash_comment	C++ style // comments are not accepted and generate errors
-prototype_errors	Controls the treatment of functions referenced or called when no prototype has been provided
-incorrect_pragma_warnings	Controls the treatment of valid #pragma directives that use the wrong syntax
-c	Stop after assembly and produce an object file for each source file
-DS32XX	Predefine S32XX as a macro, with definition 1
-DS32G2XX	Predefine S32G2XX as a macro, with definition 1
-DS32G3XX	Predefine S32G3XX as a macro, with definition 1
-DS32R45	Predefine S32R45 as a macro, with definition 1
-DGHS	Predefine GHS as a macro, with definition 1
-DUSE_SW_VECTOR_MODE	Predefine USE_SW_VECTOR_MODE as a macro, with definition 1. By default, the drivers are compiled to handle interrupts in Software Vector Mode
-DD_CACHE_ENABLE	Predefine D_CACHE_ENABLE as a macro, with definition 1. Enables data cache initialization in source file system.↵c under the Platform driver
-DI_CACHE_ENABLE	Predefine I_CACHE_ENABLE as a macro, with definition 1. Enables instruction cache initialization in source file system.c under the Platform driver
-DENABLE_FPU	Predefine ENABLE_FPU as a macro, with definition 1. Enables FPU initialization in source file system.c under the Platform driver
-DMCAL_ENABLE_USER_MODE_SUPPORT	Predefine MCAL_ENABLE_USER_MODE_SUPPORT↵RT as a macro, with definition 1. Allows drivers to be configured in user mode

3.1.2.2 GHS Assembler Options

Assembler Option	Description
-cpu=cortexm7	Selects target processor: Arm Cortex M7
-fpu=vfpv5_d16	Specifies hardware floating-point using the v5 version of the VFP instruction set, with 16 double-precision floating-point registers
-fsingle	Use hardware single-precision, software double-precision FP instructions
-preprocess_assembly_files	Controls whether assembly files with standard extensions such as .s and .asm are preprocessed

Assembler Option	Description
-list	Creates a listing by using the name and directory of the object file with the .lst extension
-c	Stop after assembly and produce an object file for each source file

3.1.2.3 GHS Linker Options

Linker Option	Description
-e Reset_Handler	Make the symbol Reset_Handler be treated as a root symbol and the start label of the application
-T linker_script_file.ld	Use linker_script_file.ld as the linker script. This script replaces the default linker script (rather than adding to it)
-map	Produce a map file
-keepmap	Controls the retention of the map file in the event of a link error
-Mn	Generates a listing of symbols sorted alphabetically/numerically by address
-delete	Instructs the linker to remove functions that are not referenced in the final executable. The linker iterates to find functions that do not have relocations pointing to them and eliminates them
-ignore_debug_references	Ignores relocations from DWARF debug sections when using -delete. DWARF debug information will contain references to deleted functions that may break some third-party debuggers
-Llibrary_path	Points to library_path (the libraries location) for thumb2 to be used for linking
-larch	Link architecture specific library
-lstartup	Link run-time environment startup routines. The source code for themodules in this library is provided in the src/libstartup directory
-lind_sd	Link language-independent library, containing support routines for features such as software floating point, run-time error checking, C99 complex numbers, and some general purpose routines of the ANSI C library
-v	Prints verbose information about the activities of the linker, including the libraries it searches to resolve undefined symbols
-nostartfiles	Controls the start files to be linked into the executable

3.2 Files required for compilation

This section describes the include files required to compile, assemble (if assembler code) and link the CAN_43_LLCE driver for S32G microcontrollers. To avoid integration of incompatible files, all the include files from other modules shall have the same AR_RELEASE_MAJOR_VERSION and AR_RELEASE_MINOR_VERSION, i.e. only files with the same AUTOSAR major and minor versions can be compiled.

CAN_43_LLCE Files

- ../Can_43_LLCE_TS_T40D11M10I5R0/include/Can_43_LLCE.h
- ../Can_43_LLCE_TS_T40D11M10I5R0/include/Can_43_LLCE_IPW.h
- ../Can_43_LLCE_TS_T40D11M10I5R0/include/Can_43_LLCE_IrqConfig.h

- ../Can_43_LLCE_TS_T40D11M10I5R0/include/Can_Llce.h
- ../Can_43_LLCE_TS_T40D11M10I5R0/include/Llce_FwVersion.h
- ../Can_43_LLCE_TS_T40D11M10I5R0/include/Llce_InterfaceCanTypes.h
- ../Can_43_LLCE_TS_T40D11M10I5R0/include/Llce_InterfaceConfig.h
- ../Can_43_LLCE_TS_T40D11M10I5R0/include/Llce_InterfaceFifo.h
- ../Can_43_LLCE_TS_T40D11M10I5R0/include/Llce_InterfaceFwMgr.h
- ../Can_43_LLCE_TS_T40D11M10I5R0/include/Llce_InterfaceIcsr.h
- ../Can_43_LLCE_TS_T40D11M10I5R0/include/Llce_RegAccess.h
- ../Can_43_LLCE_TS_T40D11M10I5R0/include/Llce_SwFifo.h
- ../Can_43_LLCE_TS_T40D11M10I5R0/include/Reg_eSys_Llce.h
- ../Can_43_LLCE_TS_T40D11M10I5R0/include/StdRegMacros.h
- ../Can_43_LLCE_TS_T40D11M10I5R0/src/Can_43_LLCE.c
- ../Can_43_LLCE_TS_T40D11M10I5R0/src/Can_43_LLCE_Irq.c
- ../Can_43_LLCE_TS_T40D11M10I5R0/src/Can_Llce.c
- ../Can_43_LLCE_TS_T40D11M10I5R0/src/Llce_InterfaceConfig.c
- ../Can_43_LLCE_TS_T40D11M10I5R0/src/Llce_SwFifo.c

CAN_43_LLCE Generated Files For driver compilation, Can_43_LLCE_[VariantName]_PBcfg.c should be generated by the user using a configuration tool. The file contains the definition of the init pointer for the respective variant.

As a deviation from standard:

- Can_43_LLCE_[VariantName]_PBcfg.c (For PB Variant) - files will contain the definition for all parameters that are variant aware, independent of the configuration class that will be selected (PC, LT,PB).
- Can_43_LLCE_Cfg.c (For PC Variant) - file will contain the definition for all configuration structures containing only variables that are not variant aware, configured and generated only once. This file alone does not contain the whole structure needed by Can_43_LLCE_Init function to configure the driver. Based on the number of variants configured in the EcuC, there can be more than one configuration structure for one module even for PreCompile variant.
- Can_43_LLCE_[VariantName]_AFcfg.c (For PC Variant) - files will contain the generated configuration structure containing advanced Llce features
- Mcu_[VariantName]_PBcfg.c - This file will contain the definition for all parameters that are variant aware, independent of the configuration class that will be selected (PC, PB) and related to Mcu driver used for clock settings
- Mcu_Cfg.c - This file will contain the definition for all configuration structures containing only variables that are not variant aware, configured and generated only once and related to Mcu driver used for clock settings.

Files from Base common folder

- ../Base_TS_T40D11M40I0R0/include/Compiler.h
- ../Base_TS_T40D11M40I0R0/include/Compiler_Cfg.h
- ../Base_TS_T40D11M40I0R0/include/ComStackTypes.h
- ../Base_TS_T40D11M40I0R0/include/Can_43_LLCE_MemMap.h
- ../Base_TS_T40D11M40I0R0/include/Mcal.h
- ../Base_TS_T40D11M40I0R0/include/Platform_Types.h
- ../Base_TS_T40D11M40I0R0/include/Std_Types.h
- ../Base_TS_T40D11M40I0R0/include/Reg_eSys.h
- ../Base_TS_T40D11M40I0R0/include/Soc_Ips.h
- ../Base_TS_T40D11M40I0R0/include/Reg_Macros.h
- ../Base_TS_T40D11M40I0R0/include/Can_GeneralTypes.h

Files from CanIf folder:

- ../CanIf_TS_T40D11M40I0R0/include/CanIf_Can.h
- ../CanIf_TS_T40D11M40I0R0/include/CanIf_Types.h.h
- ../CanIf_TS_T40D11M40I0R0/include/CanIf.h

Files from Det folder:

- ../Det_TS_T40D11M40I0R0/src/Det.c
- ../Det_TS_T40D11M40I0R0/include/Det.h

Files from EcuM folder:

- ../EcuM_TS_T40D11M40I0R0/src/EcuM.c
- ../EcuM_TS_T40D11M40I0R0/include/EcuM.h
- ../EcuM_TS_T40D11M40I0R0/include/EcuM_Cbk.h

Files from Mcu folder:

- ../Mcu_TS_T40D11M40I0R0/src/Mcu.c
- ../Mcu_TS_T40D11M40I0R0/src/Mcu_FXOSC.c
- ../Mcu_TS_T40D11M40I0R0/include/Mcu.h
- ../Mcu_TS_T40D11M40I0R0/include/Mcu_FXOSC.h

Files from Rte folder:

- ../Rte_TS_T40D11M40I0R0/src/Schm_Can_43_LLCE.c
- ../Rte_TS_T40D11M40I0R0/include/Schm_Can_43_LLCE.h
- ../Rte_TS_T40D11M40I0R0/include/Schm_Mcu.c

Files from Os folder:

- ../Os_TS_T40D11M40I0R0/src/Os_multicore.c
- ../Os_TS_T40D11M40I0R0/include/Os.h

3.3 Setting up the plugins

The CAN_43_LLCE driver was designed to be configured by using the EB Tresos Studio (version EB tresos Studio 27.1.0 b200625-0900 or later).

Location of various files inside the CAN_43_LLCE module folder:

- VSMD (Vendor Specific Module Definition) file in EB tresos Studio XDM format: /n
 - ../Can_43_LLCE_TS_T40D11M10I5R0/config/Can_43_LLCE.xdm
 - ../CanIf_TS_T40D11M40I0R0/config/CanIf.xdm
 - ../EcuM_TS_T40D11M40I0R0/config/EcuM.xdm
 - ../Base_TS_T40D11M40I0R0/config/Base.xdm
 - ../Os_TS_T40D11M40I0R0/config/Os.xdm
 - ../Mcu_TS_T40D11M40I0R0/config/Mcu.xdm
 - ../Resource_TS_T40D11M10I3R0/config/Resource.xdm
- VSMD (Vendor Specific Module Definition) file(s) in AUTOSAR compliant EPD format: /n
 - ../Can_43_LLCE_TS_T40D11M10I5R0/autosar/Can.epd
 - ../CanIf_TS_T40D11M40I0R0/autosar/CanIf.epd
 - ../EcuM_TS_T40D11M40I0R0/autosar/EcuM.epd
- Code Generation Templates for Pre-Compile time configuration parameters::
 - ../Can_43_LLCE_TS_T40D11M10I5R0/generate_PC/include/Can_43_LLCE_Cfg.h
 - ../Can_43_LLCE_TS_T40D11M10I5R0/generate_PC/include/Can_43_LLCE_Externals.h
 - ../Can_43_LLCE_TS_T40D11M10I5R0/generate_PC/src/Can_43_LLCE_Cfg.c
 - ../Can_43_LLCE_TS_T40D11M10I5R0/generate_PC/Can_43_LLCE_VersionCheck_Inc.m
 - ../Can_43_LLCE_TS_T40D11M10I5R0/generate_PC/Can_43_LLCE_VersionCheck_Src.m
- Code Generation Templates for Post-Build time configuration parameters:
 - ../Can_43_LLCE_TS_T40D11M10I5R0/generate_PB/include/Can_43_LLCE_AFcfg.h
 - ../Can_43_LLCE_TS_T40D11M10I5R0/generate_PB/src/Can_43_LLCE_PBcfg.c
 - ../Can_43_LLCE_TS_T40D11M10I5R0/generate_PB/src/Can_43_LLCE_AFcfg.c
 - ../Can_43_LLCE_TS_T40D11M10I5R0/generate_PB/Can_43_LLCE_VersionCheck_Src_PB.m

Steps to generate the configuration:

1. Copy the module folders Can_43_LLCE_TS_T40D11M10I5R0, CanIf_TS_T40D11M40I0R0, Base_TS_T40D11M40I0R0, EcuM_TS_T40D11M40I0R0, Rte_TS_T40D11M40I0R0, Mcu_TS_T40D11M40I0R0, Os_TS_T40D11M40I0R0 into the Tresos plugins folder.
2. Set the desired Tresos Output location folder for the generated sources and header files.
3. Use the EB tresos Studio GUI to modify ECU configuration parameters values.
4. Use the EB tresos Studio GUI to modify ECU configuration parameters values.
5. Generate the configuration files.

Dependencies

- RESOURCE is required to select processor derivative. Current Can_LLCE driver has support only for the derivative: s32g274a_bga525, s32g254a_bga525, s32g233a_bga525, s32g234m_bga525.
- RTE is required to provide the definitions for polling functions according to AUTOSAR 4.4 (e.g. Can_43_↔ LLCE_MainFunction_Mode()) and support for exclusive areas.
- CANIF is required for reporting status of some events.
- ECUM is required for selecting the reference to the wakeup source for every Can controller.
- DET is required for signaling the development error detection (parameters out of range, null pointers, etc).
- OS is required for providing the API used for timeout detection.
- MCU is required for checking the baudrate configured in EB Tresos.

Chapter 4

Function calls to module

- [Function Calls during Start-up](#)
- [Function Calls during Shutdown](#)
- [Function Calls during Wake-up](#)

4.1 Function Calls during Start-up

The CAN_43_LLCE module shall be initialized by Can_43_LLCE_Init() service call during the start-up. API service Can_43_LLCE_SetControllerMode(Can_Controller, CAN_CS_STARTED) shall be used for setting the CAN controller to running mode.

Note

Pin settings are not related to Can_43_LLCE driver or plugin configuration. GPIO pins used for connection of CAN_43_LLCE physical layer have to be properly assigned to the LLCE Can module prior the LLCE CAN initialization.

Because LLCE subsystem is clock gated, its clock, SRAM area and code areas must be initialized prior to the Can_43_LLCE driver initialization. The required settings may be done using the MCAL MCU module or direct writing to the relevant registers.

4.2 Function Calls during Shutdown

The LLCE Can does not support Low Power Modes, or wakeup on bus.

4.3 Function Calls during Wake-up

Wakeup is not supported.

Chapter 5

Module requirements

- Exclusive areas to be defined in BSW scheduler
- Exclusive areas not available on this platform
- Peripheral Hardware Requirements
- ISR to Configure within OS - Dependencies
- ISR Macro
- Other AUTOSAR modules - dependencies
- Data Cache Restriction
- User Mode support
- Multicore support

5.1 Exclusive areas to be defined in BSW scheduler

CAN_EXCLUSIVE_AREA_00 is used in function Can_43_LLCE_DisableControllerInterrupts to protect registers and internal variables in Low level driver layer (IPL).

CAN_EXCLUSIVE_AREA_01 is used in function Can_43_LLCE_EnableControllerInterrupts to protect registers and internal variables in Low level driver layer (IPL).

CAN_EXCLUSIVE_AREA_02 is used in function Can_43_LLCE_Write to protect internal variables in Low level driver layer (IPL).

CAN_EXCLUSIVE_AREA_02 is used in function Can_FifoTxAckNotEmptyIsr_0_7.

CAN_EXCLUSIVE_AREA_02 is used in function Can_FifoTxAckNotEmptyIsr_8_15.

CAN_EXCLUSIVE_AREA_03 is used in function Can_43_LLCE_SetControllerMode.

CAN_EXCLUSIVE_AREA_03 is used in function Can_43_LLCE_DeInit.

CAN_EXCLUSIVE_AREA_03 is used in function Can_43_LLCE_MainFunctionMode.

CAN_EXCLUSIVE_AREA_04 is used in function Can_43_LLCE_Init.

CAN_EXCLUSIVE_AREA_04 is used in function Can_43_LLCE_AbortMb.

CAN_EXCLUSIVE_AREA_04 is used in function Can_43_LLCE_SendSetBaudrateCmd.

CAN_EXCLUSIVE_AREA_04 is used in function Can_43_LLCE_CreateAfDestination.

CAN_EXCLUSIVE_AREA_04 is used in function Can_43_LLCE_DeInit.

CAN_EXCLUSIVE_AREA_04 is used in function Can_43_LLCE_GetControllerErrorState.

CAN_EXCLUSIVE_AREA_04 is used in function Can_43_LLCE_GetControllerRxErrorCounter.

Module requirements

CAN_EXCLUSIVE_AREA_04 is used in function Can_43_LLCE_GetControllerTxErrorCounter.

CAN_EXCLUSIVE_AREA_04 is used in function Can_43_LLCE_GetControllerStatus.

CAN_EXCLUSIVE_AREA_04 is used in function Can_43_LLCE_ManualBusOffRecovery.

CAN_EXCLUSIVE_AREA_04 is used in function Can_43_LLCE_RemoveFilter.

Exclusive Areas implemented in Low level driver layer (IPL)

CAN_EXCLUSIVE_AREA_00 is used in function Can_Llce_DisableControllerInterrupts to protect the updates for: - LLCE_FIFO_IER register

CAN_EXCLUSIVE_AREA_01 is used in function Can_Llce_EnableControllerInterrupts to protect the updates for: - LLCE_FIFO_IER register

CAN_EXCLUSIVE_AREA_02 is used in Can_Hth_FreeTxObject to protect the updates for Can_aul6TransmitHwObjectCnt variable and in Can_43_LLCE_SendWriteCmd to protect Can_aul6TransmitHwObjectCnt.

CAN_EXCLUSIVE_AREA_03 is used to assure no overwrites are being made on the internal variable which tracks controller states.

CAN_EXCLUSIVE_AREA_04 is used to assure that commands given to the firmware don't overlap.

Critical Region Exclusive Matrix Below is the table depicting the exclusivity between different critical region IDs from the CAN driver. If there is an “X” in a table, it means that those 2 critical regions cannot interrupt each other.

The critical regions from Tx/Rx interrupts are grouped in “ISRs Critical Regions (composed diagram)”. If an exclusive area is “exclusive” with the composed “ISRs Critical Regions (composed diagram)” group, it means that it is exclusive with each one of the ISR critical regions.

#	CAN_EXCLUSIVE_AREA_00	CAN_EXCLUSIVE_AREA_01	CAN_EXCLUSIVE_AREA_02	CAN_EXCLUSIVE_AREA_03	CAN_EXCLUSIVE_AREA_04	Interrupt Service Routines Critical Regions (composed diagram)
CAN_EXCLUSIVE_AREA_00	x					
CAN_EXCLUSIVE_AREA_01		x				
CAN_EXCLUSIVE_AREA_02			x			
CAN_EXCLUSIVE_AREA_03				x		
CAN_EXCLUSIVE_AREA_04					x	
Interrupt Service Routines Critical Regions (composed diagram)						x

5.2 Exclusive areas not available on this platform

None

5.3 Peripheral Hardware Requirements

The CAN physical interface should be connected to the CAN-LLCE module pins in order to get the CAN bus voltage levels.

There has to be at least another CAN node present on the CAN bus in order to get the CAN bus synchronized.

5.4 ISR to Configure within OS - Dependencies

ISR's used by CAN_43_LLCE The following ISR's are used by the CAN_43_LLCE driver:

ISR Name	Hardware Interrupt Vector
Can_FifoRxInNotEmptyIsr_0_7	174
Can_FifoRxInNotEmptyIsr_8_15	175
Can_FifoRxOutNotEmptyIsr_0_7	176
Can_FifoRxOutNotEmptyIsr_8_15	177
Can_FifoTxAckNotEmptyIsr_0_7	182
Can_FifoTxAckNotEmptyIsr_8_15	183

5.5 ISR Macro

RTD drivers use the ISR macro to define the functions that will process hardware interrupts. Depending on whether the OS is used or not, this macro can have different definitions.

5.5.1 Without an Operating System The macro *USING_OS_AUTOSAROS* must not be defined.

5.5.1.1 Using Software Vector Mode

The macro *USE_SW_VECTOR_MODE* must be defined and the ISR macro is defined as:

```
#define ISR(IsrName) void IsrName(void)
```

In this case, the drivers' interrupt handlers are normal C functions and their prologue/epilogue will handle the context save and restore.

5.5.1.2 Using Hardware Vector Mode

The macro `USE_SW_VECTOR_MODE` must not be defined and the ISR macro is defined as:

```
#define ISR(IsrName) INTERRUPT_FUNC void IsrName(void)
```

In this case, the drivers' interrupt handlers must also handle the context save and restore.

5.5.2 With an Operating System Please refer to your OS documentation for description of the ISR macro.

5.6 Other AUTOSAR modules - dependencies

- **Base:** The Base module contains the common files/definitions needed by all MCAL modules.
- **Mcu:** The Mcu driver provides services for basic microcontroller initialization, power down functionality, reset and microcontroller specific functions required by other MCAL software modules. The clocks need to be initialized prior to using the Can driver. The clock frequency may affect the Can bit rate, the transmitting or receiving a Can frame process.
- **Port:** The Port module is used to configure the port pins with the needed modes, before they are used by the Can_43_LLCE module.
- **EcuC :** The ECUC module is used for ECU configuration. Can_43_LLCE modules need ECUC to retrieve the variant information.
- **Det** The Det module is used for enabling Default error detection. The API function used is `Det_ReportError()`. The activation / deactivation of Default error detection is configurable using the 'CanDevErrorDetect' configuration parameter.
- **EcuM:** This module is used for processing the Wakeup notifications of CAN. Whenever the module is in 'Sleep' mode and a wakeup event occurs, it is reported to EcuM through the `EcuM_CheckWakeupEvent()` API.
- **Rte:** This module is needed for implementing data consistency of exclusive areas that are used by Can_43_LLCE module.
- **Os:** The Os module provides services for configuring timer/software ticks necessary in Can_43_LLCE driver for timeout configuring.

5.7 Data Cache Restriction

None.

5.8 User Mode support

- [User Mode configuration in the module](#)
- [User Mode configuration in AutosarOS](#)

5.8.1 User Mode configuration in the module

The CAN_LLCE driver can be run in user mode if the following steps are performed:

- Enable CanEnableUserModeSupport from the configuration and MCAL_ENABLE_USER_MODE_SUPPORT is defined at compilation time. CAN_LLCE module will perform a trusted call to the Llce_Sema42_Lock and Llce_Sema42_Unlock functions.

5.8.2 User Mode configuration in AutosarOS

When User mode is enabled, the driver may have the functions that need to be called as trusted functions in AutosarOS context. Those functions are already defined in driver and declared in the header <IpName>_Ip_TrustedFunctions.h. This header also included all headers files that contains all types definition used by parameters or return types of those functions. Refer the chapter [User Mode configuration in the module](#) for more detail about those functions and the name of header files they are declared inside. Those functions will be called indirectly with the naming convention below in order to AutosarOS can call them as trusted functions.

```
Call_<Function_Name>_TRUSTED(parameter1,parameter2,...)
```

That is the result of macro expansion OsIf_Trusted_Call in driver code:

```
#define OsIf_Trusted_Call[1-6params](name,param1,...,param6) Call_##name##_TRUSTED(param1,...,param6)
```

So, the following steps need to be done in AutosarOS:

- Ensure MCAL_ENABLE_USER_MODE_SUPPORT macro is defined in the build system or somewhere global.
- Define and declare all functions that need to call as trusted functions follow the naming convention above in Integration/User code. They need to be visible in Os.h for the driver to call them. They will do the marshalling of the parameters and call CallTrustedFunction() in OS specific manner.
- CallTrustedFunction() will switch to privileged mode and call TRUSTED_<Function_Name>().
- TRUSTED_<Function_Name>() function is also defined and declared in Integration/User code. It will un-marshalling of the parameters to call <Function_Name>() of driver. The <Function_Name>() functions are already defined in driver and declared in <IpName>_Ip_TrustedFunctions.h. This header should be included in OS for OS call and indexing these functions.

See the sequence chart below for an example calling Linflexd_Uart_Ip_Init_Privileged() as a trusted function.

Module requirements

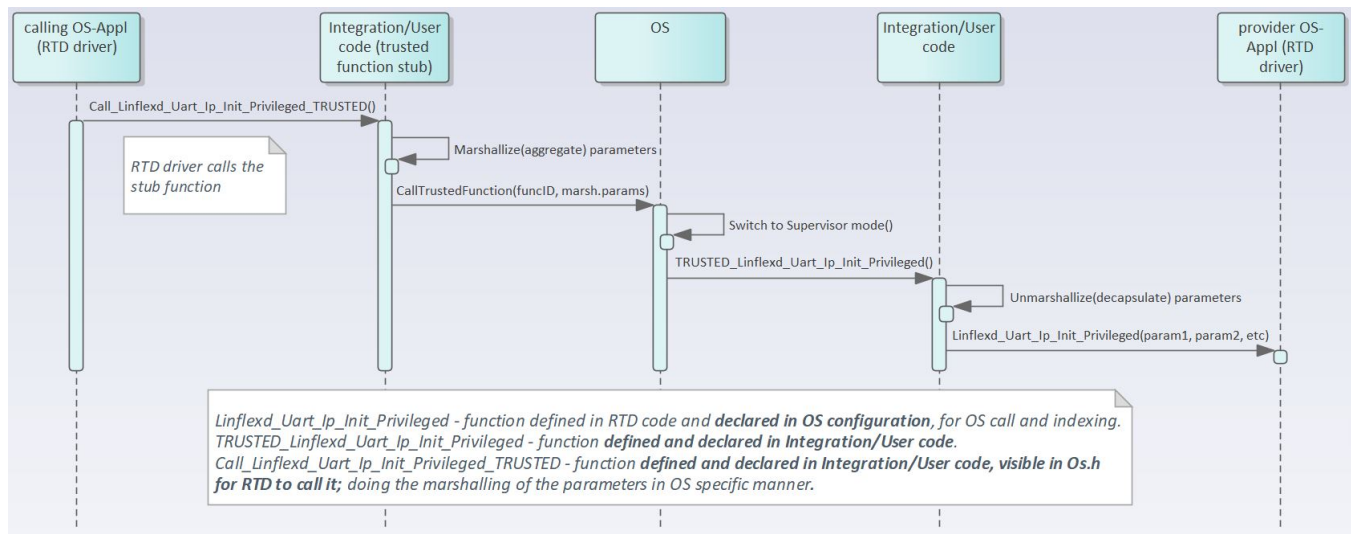


Figure 5.1 Example sequence chart for calling `Linflexd_Uart_Ip_Init_Privileged` as trusted function

5.9 Multicore support

The driver implements the "Autosar 4.4 MCAL Multicore Distribution" according to type II, in which the mappable element is set to HW Controller. For additional details, please refer to AUTOSAR_EXP_BSWDistributionGuide. The driver and the mappable elements can be allocated to zero, one or two ECUC partitions, by means of "`CanEcucPartionRef`". If the driver is mapped to zero ECUC partitions, the driver behavior reverts to single-core implementation, similar to previous Autosar versions. If the driver is mapped to one or two ECUC partitions, the driver enforces the following multi-core assumptions:

- The driver assumes there is a single `EcucPartition` allocated per core. Internally, the module will use the Core ID returned by `GetCoreID` API to reference the appropriate global data and configuration elements.
- The driver assumes the `EcucCoreIDs` are defined in a compact/consecutive order, starting from zero. The rationale is that the number of `EcucPartitions` is used for dimensioning the Can internal variables and the `EcucCoreIDs` are used for indexing those variables. (AR-86601 Zero based and dense IDs for OS-Cores and OSApplications) The driver assumes that initialization is performed on each core, `Can_43_LLCE_Init()` is called separately for each core, using a different configuration structure (Type II). The Can initialization expects the upper layer will pass the correct initialization pointer, specific to the partition in which the driver is to be used. For example: `EcucPartition_1` is assigned to CoreID 1; `Can_43_LLCE_Init` function will be called with `Can_43_LLCE_Config_EcucPartition_1` configuration structure, on Core 1.
- The driver will check upon each API call if the requested resource is configured to be available on the current core, if DET error reporting is enabled.
- The driver requires that all variables in NonCacheable MemMap sections be allocated accordingly, to avoid data corruption in multicore context.
- The driver assumes that RTE module implements the EXCLUSIVE AREAS to be core-aware only. The rationale is that the module implementation ensures data integrity by separating the mappable elements for different cores already, thus implementing the EXCLUSIVE AREAS in a blocking manner (ex: spin-lock) on

a multicore scope, might affect the performance of the drivers on the two cores, although they might access separate HW elements. For single-core scope, the EXCLUSIVE AREAS keep the same purpose as on previous AUTOSAR implementations.

- The driver assumes that each interrupt is routed by the system only to the core on which is supposed to be serviced. The configuration structure name shall be available in the caller scope of Can_Init function by being declared with EXTERN, according to its generated name.

If CanEcucPartionRef is used, each partition is allocated a LLCE host interface (HIF) based on the order of elements in CanEcucPartionRef. This results in only 2 partition references being possible for the Can_43_LLCE driver. The order may be relevant when sharing use of LLCE with a non-AUTOSAR application.

Note: when Advanced Features (Can2Can or Can2Eth routing, logging or custom processing) are used in a multicore setup, a kind of race condition happens on creating the destinations. This is solved by having the core that owns LLCE HIF0 create the destinations, and the core that owns LLCE HIF1 waiting for the other core to set a flag. This creates a forced synchronization point inside the Can_43_LLCE driver.

Chapter 6

Main API Requirements

- [Main function calls within BSW scheduler](#)
- [API Requirements](#)
- [Calls to Notification Functions, Callbacks, Callouts](#)

6.1 Main function calls within BSW scheduler

The CAN_43_LLCE Driver support 5 main functions that can be configured to be scheduled by BSW scheduler:

- *void Can_43_LLCE_MainFunction_Write(void)*
- *void Can_43_LLCE_MainFunction_Read(void)*
- *void Can_43_LLCE_MainFunction_BusOff(void)*
- *void Can_43_LLCE_MainFunction_Mode(void)*
- *void Can_43_LLCE_MainFunction_ErrorNotification(void)*

These Autosar APIs are scheduled if these 3 events are configured to be in “Polling” mode by the following parameters:

- CanTxProcessing
#define CAN_43_LLCE_TXPOLL_SUPPORTED (STD_ON)
- CanRxProcessing
#define CAN_43_LLCE_RXPOLL_SUPPORTED (STD_ON)
- CanBusoffProcessing
#define CAN_43_LLCE_BUSOFFPOLL_SUPPORTED (STD_ON)

The period for polling is configured by the following 4 parameters:

- CanMainFunctionWritePeriod
#define CAN_43_LLCE_MAINFUNCTION_PERIOD_WRITE (uint32)0.0010U
- CanMainFunctionReadPeriod
#define CAN_43_LLCE_MAINFUNCTION_PERIOD_READ (uint32)0.0010U
- CanMainFunctionBusoffPeriod
#define CAN_43_LLCE_MAINFUNCTION_PERIOD_BUSOFF (uint32)0.0010U
- CanMainFunctionModePeriod
#define CAN_43_LLCE_MAINFUNCTION_MODE_PERIOD (uint32)0.0010U

Note

A configuration for a hardware unit can be possible in such a way that one controller can handle events by interrupt mode and by polling mode simultaneously (mixed mode).

6.2 API Requirements

CAN360,CAN361,CAN362,CAN363,CAN228,CAN112,CAN185,CAN186,CAN235

6.3 Calls to Notification Functions, Callbacks, Callouts

Call-back Notifications

The CAN_LLCE driver provides the following call-back notifications:

- **CanIf_TxConfirmation:** This CAN Interface call-back function is called when a CAN message has been transmitted.
void CanIf_TxConfirmation(PduIdType CanTxPduId)
- **CanIf_RxIndication:** This CAN Interface call-back function is called when valid CAN message is received.
void CanIf_RxIndication(const Can_HwType* Mailbox, const PduInfoType* PduInfoPtr)
- **CanIf_ControllerBusOff:** This CAN Interface call-back function is called when the CAN controller reached the bus-off state (see CAN specification for further details).
void CanIf_ControllerBusOff(uint8 Controller)
- **CAN_43_LLCE_ErrorNotificationCallback:** This CAN configurable callback for error notification, according to the configuration of possible error sources in CanErrorReporting container.
void CAN_43_LLCE_ErrorNotificationCallback(CanErrorNotificationType* error)
- **CAN_43_LLCE_TxTimestampNotification:** This CAN configurable callback for received frame timestamp.
void CAN_43_LLCE_RxTimestampNotification(Can_HwHandleType Hoh, uint32 u32TimestampVal);
- **CAN_43_LLCE_RxTimestampNotification:** This CAN configurable callback for transmitted frame timestamp.
void CAN_43_LLCE_TxTimestampNotification(Can_HwHandleType Hoh, PduIdType CanTxPduId, uint32 u32TimestampVal);
- **CAN_43_LLCE_ControllerBusOffCallback:** This is a configurable callback which is called in case of bus-off recovery is set to Auto-recovery or Manual-recovery (non Autosar).
void CAN_43_LLCE_ControllerBusOffCallback(uint8 Controller);
- **CAN_43_LLCE_CanAuxFilterCallback:** This is a configurable callback which is called in case of CanAuxFilter container is populated with entries to be searched using the RXLUT2 search engine in LLCE fw.
void CAN_43_LLCE_CanAuxFilterCallback(uint16 u16FilterId, uint16 u16AuxSearchResult);

User Notification

- None

Chapter 7

Memory allocation

- [Sections to be defined in MemMap.h](#)
- [Linker command file](#)

7.1 Sections to be defined in MemMap.h

Section name	Type of section	Description
CAN_43_LLCE_START_SEC_CODE	Code	Start of memory Section for Code.
CAN_43_LLCE_STOP_SEC_CODE	Code	End of above section.
CAN_43_LLCE_START_SEC_CONFIG_DATA_UN↔ SPECIFIED	Configuration Data	Start of Memory Section for Config Data.
CAN_43_LLCE_STOP_SEC_CONFIG_DATA_UN↔ SPECIFIED	Configuration Data	End of above section.
CAN_43_LLCE_START_SEC_CONSTANT_UN↔ SPECIFIED	Constant Data	Used for constants
CAN_43_LLCE_STOP_SEC_CONSTANT_UN↔ SPECIFIED	Constant Data	End of above section.
CAN_43_LLCE_START_SHARED_MEMORY↔	Variables	Used for sharing data with LLCE
CAN_43_LLCE_STOP_SHARED_MEMORY↔	Variables	End of above section.
CAN_43_LLCE_START_SEC_VARIABLES↔ INIT_UNSPECIFIED	Variables	Used for variables, structures, arrays, when the SIZE (alignment) does not fit the criteria of 8, 16 or 32 bit. These variables are initialized with values after every reset.
CAN_43_LLCE_STOP_SEC_VARIABLES↔ INIT_UNSPECIFIED	Variables	End of above section.
CAN_43_LLCE_START_SEC_VARIABLES↔ NEVER_CLEARED_UNSPECIFIED	Variables	These variables are never cleared and never initialized by start-up code.
CAN_43_LLCE_STOP_SEC_VARIABLES↔ NEVER_CLEARED_UNSPECIFIED	Variables	End of above section.

7.2 Linker command file

Memory shall be allocated for every section defined in the driver's "<Module>__MemMap.h.

In addition, some sections are intended to be placed in Shared RAM, for communication with LLCE Firmware. These sections shall be placed in the linker command file in exactly the manner specified. The host application shall not access at all next sections before to be configured(set clock tree, etc.) e.g. Do not define initialized global variable inside shared memory which are initialized by application startup code. For example, this can be done by specifying '(NOLOAD)' directive when using GCC compiler, in order to avoid loading into memory at runtime by any access before LLCE firmware is initialized.

Section in linker file	Start address	Size (bytes)
can_43_llce_sharedmemory	0x43800000	0x3C800
llce_boot_end	0x4383C8A0	0x50



Chapter 8

Integration Steps

This section gives a brief overview of the steps needed for integrating this module:

1. Generate the required module configuration(s). For more details refer to section [Files Required for Compilation](#)
2. Allocate the proper memory sections in the driver's memory map header file ("`<Module>_MemMap.h`") and linker command file. For more details refer to section [Sections to be defined in `<Module>\_MemMap.h`](#)
3. Compile & build the module with all the dependent modules. For more details refer to section [Building the Driver](#)

Chapter 9

External assumptions for driver

The section presents requirements that must be complied with when integrating CAN_43_LLCE driver into the application.

External Assumption Req ID	External Assumption Text
SWS_Can_00436	Can_GeneralTypes.h shall contain all types and constants that are shared among the AUTOSAR CAN modules Can, CanIf and CanTrcv. Note: Implemented in base
SWS_Can_00415	Name: - Can_PduType - Type: - Structure - Element: - PduIdType - swPduHandle - - - uint8 - length - - - Can_IdType - id - - - uint8* - sdu - - - Description: - This type unites PduId (swPduHandle), SduLength (length), SduData (sdu), and CanId (id) for any CAN L-SDU. - Available via: - Can_GeneralTypes.h - Note: defined into Can_GeneralTypes.h included into Base module
SWS_Can_00416	Name: - Can_IdType - Type: - uint32 - Range: - Standard32Bit - - - 0..0x400007FF - Extended32Bit - - - 0..0xDFFFFFFF - Description: - Represents the Identifier of an L-PDU. The two most significant bits specify the frame type: 00 CAN message with Standard CAN ID 01 CAN FD frame with Standard CAN ID 10 CAN message with Extended CAN ID 11 CAN FD frame with Extended CAN ID - Available via: - Can_GeneralTypes.h - Note: defined into Can_GeneralTypes.h included into Base module
SWS_Can_00429	Name: - Can_HwHandleType - Type: - uint8, uint16 - Range: - Standard - - - 0..0x0FFF - Extended - - - 0..0xFFFF - Description: - Represents the hardware object handles of a CAN hardware unit. For CAN hardware units with more than 255 HW objects use extended range. - Available via: - Can_GeneralTypes.h - Note: defined into Can_GeneralTypes.h included into Base module
SWS_Can_00039	Range: - - 0x02 - transmit request could not be processed because no transmit object was available - Description: - Overlayed return value of Std_ReturnType for CAN driver API Can_Write() - Available via: - Can_GeneralTypes.h - Note: defined into Can_GeneralTypes.h included into Base module
SWS_Can_00024	The valid values that can be configured are hardware dependent. Therefore the rules and constraints can't be given in the standard. The configuration tool is responsible to do a static configuration checking, also regarding dependencies between modules (i.e. Port driver, MCU driver etc.)
EA_RTD_00071	If interrupts are locked, a centralized function pair to lock and unlock interrupts shall be used.

External assumptions for driver

External Assumption Req ID	External Assumption Text
EA_RTD_00081	The integrator shall assure that <MSN>_Init() and <MSN>_DeInit() functions do not interrupt each other.
EA_RTD_00082	When caches are enabled and data buffers are allocated in cacheable memory regions the buffers involved in DMA transfer shall be aligned with both start and end to cache line size. Note: Rationale: This ensures that no other buffers/variables compete for the same cache lines.
EA_RTD_00092	The integrator shall allocate a single EcucPartition per core or the partition in which the Can_llce is allocated shall be exclusively mapped to a core. Note: Internally, the Can_llce will use the Core ID returned by GetCoreID API to reference the appropriate global data and configuration elements, that is why a core should reference only one configured partition.
EA_RTD_00093	The application shall define EcucCoreIDs in a compact/consecutive order, starting from zero.
EA_RTD_00094	When multicore support is enabled, the application shall call Can_llce_↵Init() for each core, using the dedicated configuration pointer for that core.
EA_RTD_00096	The application shall pass the correct initialization pointer, specific to the partition in which the driver is to be used.
EA_RTD_00113	When RTD drivers are integrated with AutosarOS and User mode support is enabled, the integrator shall assure that the definition and declaration of all RTD functions needed to be called as trusted functions follow the naming convention Call<Function_Name>TRUSTE↵D(parameter1,parameter2,...) in Integration/User code. They need to visible in Os.h for the driver to call them. They will call RTD <Function_↵Name>() as trusted functions in OS specific manner.

How to Reach Us:

Home Page:

nxp.com

Web Support:

nxp.com/support

Information in this document is provided solely to enable system and software implementers to use NXP products. There are no express or implied copyright licenses granted hereunder to design or fabricate any integrated circuits based on the information in this document. NXP reserves the right to make changes without further notice to any products herein.

NXP makes no warranty, representation, or guarantee regarding the suitability of its products for any particular purpose, nor does NXP assume any liability arising out of the application or use of any product or circuit, and specifically disclaims any and all liability, including without limitation consequential or incidental damages. "Typical" parameters that may be provided in NXP data sheets and/or specifications can and do vary in different applications, and actual performance may vary over time. All operating parameters, including "typicals," must be validated for each customer application by customer's technical experts. NXP does not convey any license under its patent rights nor the rights of others. NXP sells products pursuant to standard terms and conditions of sale, which can be found at the following address: nxp.com/SalesTermsandConditions.

NXP, the NXP logo, NXP SECURE CONNECTIONS FOR A SMARTER WORLD, COOLFLUX, EMBRACE, GREENCHIP, HITAG, I2C BUS, ICODE, JCOP, LIFE VIBES, MIFARE, MIFARE CLASSIC, MIFARE DESFire, MIFARE PLUS, MIFARE FLEX, MANTIS, MIFARE ULTRALIGHT, MIFARE4MOBILE, MIGLO, NTAG, ROADLINK, SMARTLX, SMARTMX, STARPLUG, TOPFET, TRENCHMOS, UCODE, Freescale, the Freescale logo, Altivec, C-5, CodeTEST, CodeWarrior, ColdFire, ColdFire+, C-Ware, the Energy Efficient Solutions logo, Kinetis, Layerscape, MagniV, mobileGT, PEG, PowerQUICC, Processor Expert, QorIQ, QorIQ Qonverge, Ready Play, SafeAssure, the SafeAssure logo, StarCore, Symphony, VortiQa, Vybrid, Airfast, BeeKit, BeeStack, CoreNet, Flexis, MXC, Platform in a Package, QUICC Engine, SMARTMOS, Tower, TurboLink, and UMEMS are trademarks of NXP B.V. All other product or service names are the property of their respective owners. ARM, AMBA, ARM Powered, Artisan, Cortex, Jazelle, Keil, SecurCore, Thumb, TrustZone, and Vision are registered trademarks of ARM Limited (or its subsidiaries) in the EU and/or elsewhere. ARM7, ARM9, ARM11, big.LITTLE, CoreLink, CoreSight, DesignStart, Mali, mbed, NEON, POP, Sensinode, Socrates, ULINK and Versatile are trademarks of ARM Limited (or its subsidiaries) in the EU and/or elsewhere. All rights reserved. Oracle and Java are registered trademarks of Oracle and/or its affiliates. The Power Architecture and Power.org word marks and the Power and Power.org logos and related marks are trademarks and service marks licensed by Power.org.

© 2023 NXP B.V.

